xCurl 1.1.0

Manual práctico de integración, flujo y contratos

Paso a paso, sin dar por supuesto conocimiento interno de la librería

Campo	Valor
Versión	1.1.0 estable
PHP soportado	^8.5.12
Fecha de metadatos de release	25/10/2026
Distribución	Composer/Packagist recomendado; alternativas GitHub VCS y standalone
Repositorio fuente	https://github.com/absolutoffice/xcurl
Documento	xCurl-1.1.0-Manual-practico-es.pdf
Base de verificación	Código 1.1.0, metadatos y regresiones incluidas revisados el 25/10/2026

Para quién es este manual

Para usuarios que necesitan instalar, integrar, operar o diagnosticar xCurl sin deducir su comportamiento leyendo toda la implementación. Las afirmaciones históricas de QA para mantenedores no se presentan como evidencia actual de la release.

Absolut Office - documentación para el usuario

0. Si solo lees cinco minutos

La idea completa de xCurl en una sola página.

Regla mental principal

xCurl tiene dos capas. Un TRANSPORTE realiza exactamente una petición HTTPS. HttpClient decide redirects, retries, Retry-After y el deadline global. HttpTransportFactory solo selecciona un backend según las capacidades disponibles en el runtime.

Pregunta	Respuesta corta
¿Cómo lo instalo?	Composer desde Packagist: composer require absolutoffice/xcurl:^1.1.
¿Qué API uso normalmente?	HttpClient::send(HttpRequest, RequestOptions).
¿Cómo se elige backend?	HttpTransportFactory::create() o createStrict().
¿Hay failover si falla una petición?	No. El fallback solo ocurre al seleccionar por capacidades del runtime.
¿Qué se reintenta por defecto?	GET/HEAD; errores transitorios, 408, 429 y 5xx; máximo dos retries.
¿Quién sigue redirects?	Solo HttpClient. El transporte devuelve 3xx sin seguirlo.
¿Body null y vacío son iguales?	No. null = sin entidad; cadena vacía = entidad explícita de 0 bytes.
¿Cómo descargo a fichero?	RequestOptions(targetPath: ...). El target no debe existir.
¿Cómo diagnostico el backend?	HttpTransportFactory::diagnostics().
¿Cómo verifico el runtime instalado?	composer check-platform-reqs --no-dev, diagnostics() y una petición HTTPS simple.

Mapa del manual

Caso	Motivo
1-4	Modelo mental, instalación, primera petición y selección de transport.
5-8	Contratos de request/response, descargas y errores tipados.
9-13	Retries, redirects, tiempos, TLS/límites y diagnóstico.
14-16	Verificación actual, distribución Composer y mapa de API pública.
17-22	Recetas, anti-patrones, troubleshooting, despliegue, mantenimiento y referencias.
Apéndice	Chuleta de una página.

1. Modelo mental: qué hace cada capa

Decide primero si necesitas una petición de bajo nivel o una operación lógica con política.

Concepto	Qué significa en xCurl	Qué NO significa
Fallback	Elegir cURL, streams HTTPS o socket según capacidades actuales del runtime.	Probar otro backend después de un fallo DNS/TLS/lectura.
Retry	Repetir una petición lógica según RetryPolicy usando el mismo transport seleccionado.	Cambiar automáticamente de transport.
Redirect	HttpClient sigue Location y aplica reglas de método/body/credenciales.	Que el transport siga redirects por su cuenta.

Frase para recordar

Factory elige el motor. Transport hace un intento. HttpClient organiza la operación.

Caso

Caso	Usa	Motivo
Una petición, sin política retry/redirect	HttpTransportInterface::request()	Contrato pequeño y predecible.
Uso normal de aplicación	HttpClient::send()	Separa datos del request y política, y conserva historial.
Llamada escalar de compatibilidad	HttpClient::request()	Wrapper compatible sobre send().
Diagnóstico/test determinista de backend	HttpTransportFactory::createNamed()	No es la selección normal de negocio.

2. Instalación y runtime

Composer es la vía recomendada para consumidores en 1.1.0. El modo standalone sigue disponible como fallback de compatibilidad.

Recomendado: Composer / Packagist

Instala la línea 1.1 en el proyecto consumidor. Composer aplica el requisito de PHP y proporciona autoload PSR-4.

```
composer require absolutoffice/xcurl:^1.1
```

Alternativa directa por GitHub VCS

Úsala cuando quieras fijar deliberadamente el tag estable de GitHub como repositorio VCS en lugar de la resolución normal de Packagist.

```
composer config repositories.xcurl vcs https://github.com/absolutoffice/xcurl.git
composer require absolutoffice/xcurl:1.1.0 --no-plugins --no-scripts
```

Bootstrap de la aplicación

Carga una sola vez el autoloader del proyecto consumidor. No ejecutes una instalación Composer separada dentro de vendor/absolutoffice/xcurl.

```
require_once __DIR__ . '/vendor/autoload.php';

use AbsolutOffice\XCurl\HttpTransportFactory;

$transport = HttpTransportFactory::create();
```

Nota de producción

El servidor de producción necesita el árbol vendor desplegado y su autoloader generado. El ejecutable composer puede quedarse en CI/build si las dependencias se instalan antes del despliegue.

Fallback standalone

El repositorio conserva un autoloader local ligero. Úsalo solo cuando el despliegue Composer no sea adecuado; el autoloader standalone no realiza los platform checks de Composer.

```
require_once '/path/to/xcurl/autoload.php';
```

3. Primera petición: camino recomendado

Usa HttpClient cuando la operación pueda necesitar redirects, retries, deadline o historial de intentos.

```
use AbsolutOffice\XCurl\Client\HttpClient;
use AbsolutOffice\XCurl\Client\RequestOptions;
use AbsolutOffice\XCurl\HttpRequest;
use AbsolutOffice\XCurl\HttpTransportFactory;

$client = new HttpClient(HttpTransportFactory::create());
$result = $client->send(
    new HttpRequest(
        'GET',
        'https://example.com/data.json',
        ['User-Agent: MyProject/1.0'],
    ),
    new RequestOptions(
        timeoutSeconds: 30,
        deadlineSeconds: 90,
    ),
);

$response = $result->response;
```

Qué ocurre

1. La factory mira las capacidades del SAPI actual y crea un solo transport.
2. HttpClient recibe el request inmutable y las opciones de ejecución.
3. El transport valida método, URL, headers, body, límites y target cuando es posible antes de I/O de red.
4. El transport seleccionado ejecuta una sola petición HTTPS.
5. HttpClient decide si devuelve, redirige o reintenta.
6. El resultado incluye respuesta final, URL final, contadores e historial tipado.

Respuesta HTTP != éxito de negocio

Un 404, 409 o 500 sigue siendo una respuesta HTTP. xCurl no lanza una excepción solo por ser 4xx/5xx; la política puede reintentar ciertos status y la aplicación decide el significado final.

4. Selección de transport y fallback

La selección depende de capacidades disponibles, no de si una petición concreta falla después.

Modo	Orden de selección	Cuándo usarlo
create()	cURL -> streams HTTPS -> socket TLS	Compatibilidad general; ruta por defecto.
createStrict()	cURL -> socket TLS	Cuando xCurl debe aplicar por sí mismo framing de respuesta más estricto.

No existe failover por request

Si create() selecciona cURL y luego la petición falla por DNS, timeout, TLS o lectura, xCurl no prueba silenciosamente streams o socket. HttpClient puede reintentar el mismo transport si la política lo permite.

Síntoma

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

5. Contrato de request: lo que entra

Todos los transports incluidos comparten la misma validación para aceptar o rechazar igual la misma entrada.

Campo	Regla	Ejemplo
Método	Token HTTP válido; se normaliza a mayúsculas; CONNECT se rechaza.	get -> GET; CONNECT -> INVALID_REQUEST
HEAD	Se rechaza un body explícito, incluso cadena vacía.	HEAD + body "" -> INVALID_REQUEST
URL	Solo HTTPS absoluta; URI ASCII RFC 3986; el no-ASCII debe venir percent-encoded; se rechaza userinfo.	https://example.com/a%20b
Headers	Sin controles inseguros; nombre token; Host duplicado se rechaza.	Host puede aparecer una vez y debe ser autoridad válida.
Transfer-Encoding	No soportado en request.	Usa framing Content-Length exacto.
Content-Length	Si viene, una sola vez y coincide con bytes del body; si no, xCurl lo sintetiza para body explícito.	"abc" -> 3 bytes

Semántica del body

Valor	Significado	Content-Length
null	No existe entidad de request.	No se sintetiza Content-Length; si el caller lo declara solo puede ser 0.
""	Entidad explícita de cero bytes.	Se sintetiza Content-Length: 0 si falta.
"abc"	Entidad binaria de 3 bytes.	Se sintetiza Content-Length: 3 si falta.

Límite de request

maxRequestBytes es opcional. Si se supera, falla antes de red con TransportError::REQUEST_LIMIT no transitorio. Un límite configurado menor que 1 byte es inválido.

```
$transport = HttpTransportFactory::create(
    maxResponseBytes: 64 * 1024 * 1024,
    maxRequestBytes: 8 * 1024 * 1024,
);
```

6. Contrato de response: lo que sale

Cada transport incluido devuelve un HttpResponse normalizado.

Campo	Significado
statusCode	Status HTTP final, 100..599.
headers	Bloque final normalizado; puede incluir trailers si el backend los expone.
body	Body en memoria cuando no se usa target file.
bytes	Bytes de body recibidos o escritos.
targetCreated	true solo cuando ese intento completado creó el target de forma exclusiva.

Semántica de no-body

HEAD y status 204, 205 y 304 son respuestas sin body. 101 Switching Protocols no está soportado. Una respuesta que termina únicamente en un 1xx informativo es error de protocolo.

Framing estricto

Caso	Significado
Transfer-Encoding + Content-Length	Se rechaza por ambigüedad.
Transfer-Encoding distinto de chunked	Se rechaza.
Content-Length conflictivo	Se rechaza.
Body más corto que Content-Length	Error READ transitorio.
Streams HTTPS de PHP	Backend de compatibilidad; el wrapper puede ocultar Transfer-Encoding, por eso createStrict() lo excluye.

7. Descargas a fichero y limpieza segura por propiedad

1.1.0 hace explícita la propiedad del fichero de descarga. El target no es un destino de sobrescritura.

```
$tmp = sys_get_temp_dir() . '/download-' . bin2hex(random_bytes(8)) . '.tmp';

$result = $client->send(
    new HttpRequest('GET', 'https://example.com/file.dat'),
    new RequestOptions(
        timeoutSeconds: 30,
        targetPath: $tmp,
        deadlineSeconds: 120,
    ),
);

// Validate size/checksum/business rules, then promote the file.
```

- Con HttpClient, targetPath no debe existir; ficheros y symlinks existentes se rechazan antes de la primera petición.
- Los transports incluidos crean targets de body de forma exclusiva. Las respuestas sin body no crean target.
- Si falla un transport, el transport elimina el parcial que él haya creado.
- Antes de redirect o retry por response, HttpClient elimina solo el intermedio cuyo response completado indica targetCreated=true.
- Si aparece un fichero y el intento completado no reclama su propiedad, la continuación se detiene con ClientError::TARGET_CLEANUP y no se borra el fichero.
- Usa un temporal nuevo por operación; valida checksum/tamaño/reglas de negocio y después renómbralo o promuévelo en la aplicación.

8. Errores: cómo saber qué ha pasado

No uses mensajes de excepción para decidir lógica. Usa enums, transient e historial del cliente.

Errores del transport

TransportError	value
DNS	dns
CONNECT	connect
TLS	tls
TIMEOUT	timeout
READ	read
WRITE	write
PROTOCOL	protocol
RESPONSE_LIMIT	response_limit
REQUEST_LIMIT	request_limit
INVALID_REQUEST	invalid_request
UNAVAILABLE	unavailable
UNKNOWN	unknown

Errores del cliente

ClientError	Significado
INVALID_REDIRECT	Destino de redirect múltiple, inválido o no HTTPS.
REDIRECT_LIMIT	Se superó el máximo de redirects seguidos.
DEADLINE	No queda tiempo para redirect, retry o nuevo intento.
TARGET_CLEANUP	No se puede continuar de forma segura por propiedad/limpieza del target intermedio.
TRANSPORT	Fallo de transport no reintentado o condiciones de retry agotadas.

transient manda

TransportError clasifica la categoría. HttpTransportException::$transient indica si repetir el mismo transport puede razonablemente funcionar. RetryPolicy además comprueba método y presupuesto de retries.

```
try {
    $result = $client->send($request, $options);
} catch (AbsolutOffice\XCurl\Client\HttpClientException $e) {
    error_log('client_error=' . $e->error->value);
    foreach ($e->history as $attempt) {
        error_log($attempt->action->value);
    }
}
```

9. Retries: cuándo repite xCurl

La política por defecto es conservadora para evitar duplicar efectos laterales.

Parámetro	Default	Efecto
maxRetries	2	Máximo dos repeticiones después del primer intento.
backoffMilliseconds	[1000, 3000]	Delay de retry #1 y #2; el último valor se reutiliza si hace falta.
methods	[GET, HEAD]	Solo estos métodos son retryables automáticamente por defecto.
statusCodes	[408, 429]	Status retryables explícitos.
retryServerErrors	true	También reintenta 500..599.
maxRetryAfterSeconds	60	Tope de un Retry-After válido.

POST no se reintenta por defecto

Si habilitas POST/PUT/PATCH en RetryPolicy, la idempotencia es responsabilidad de tu aplicación/API. xCurl no puede saber si repetir una operación con efectos laterales es seguro.

10. Redirects: reglas exactas

HttpClient sigue redirects manualmente; los transports nunca activan seguimiento automático.

Status	Método siguiente	Body	Headers de entidad
301	preserva	preserva	preserva
302	preserva	preserva	preserva
303	GET salvo HEAD	elimina si cambia método	elimina Content-Length y Content-Type si cambia método
307	preserva	preserva	preserva
308	preserva	preserva	preserva

Reglas de seguridad de headers

- Cada redirect seguido elimina un Host explícito para regenerar la autoridad.
- Los redirects cross-origin eliminan Authorization por defecto.
- Los redirects cross-origin eliminan Cookie por defecto.
- Ambos comportamientos son configurables en RedirectPolicy.
- Múltiples Location son inválidos mientras el seguimiento está habilitado.
- maxRedirects: 0 significa no seguir; el 3xx se devuelve sin cambios.
- La continuación de redirects sigue siendo solo HTTPS.

11. Timeout y deadline no son lo mismo

timeoutSeconds protege un intento; deadlineSeconds limita toda la operación HttpClient.

Concepto	Qué significa en xCurl	Efecto
timeoutSeconds	Un intento de transport.	Safety timeout con semántica específica del backend.
deadlineSeconds	Operación HttpClient completa.	Incluye retries, redirects y sleeps; limita cada intento nuevo.

Backend

Backend	Uso de timeoutSeconds
cURL	Timeout total del request y cap de conexión.
Streams HTTPS	Timeout nativo de I/O + comprobación timed_out después de lecturas.
Socket TLS	Conexión TCP, I/O del handshake TLS y posteriores operaciones de stream.

Comportamiento del deadline

Antes de cada nuevo intento HttpClient calcula el tiempo restante. Si queda menos de un segundo o no cabe el delay más otro intento, falla con ClientError::DEADLINE.

12. TLS, CA privada y límites

La verificación TLS de peer y hostname permanece activada. Un CA bundle personalizado cambia las anclas de confianza, no la política de verificación.

```
$caFile = __DIR__ . '/config/private-ca.pem';

$curl    = new CurlHttpTransport($caFile);
$streams = new StreamHttpTransport($caFile);
$socket  = new SocketHttpTransport($caFile);
```

Regla	Significado
Verificación de peer	Activa.
Verificación de hostname	Activa.
Ruta CA vacía/NUL	Configuración inválida.
maxResponseBytes < 1	Configuración inválida.
maxRequestBytes < 1	Configuración inválida.
Response supera límite	TransportError::RESPONSE_LIMIT, no transitorio; se elimina parcial propio.
Request supera límite	TransportError::REQUEST_LIMIT, no transitorio; se rechaza antes de red.

```
$transport = HttpTransportFactory::create(
    maxResponseBytes: 64 * 1024 * 1024,
    maxRequestBytes: 8 * 1024 * 1024,
);
```

13. Diagnóstico: qué puede usar cada SAPI

CLI y Apache/FPM pueden ejecutar la misma versión de PHP con php.ini, extensiones y capacidades distintas. Comprueba diagnostics() en el SAPI que hace la petición.

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

Síntoma	Comprobar
cURL=false	Comprueba ext-curl y funciones cURL requeridas.
streams=false	Comprueba allow_url_fopen y wrapper https.
socket=false	Comprueba ext-openssl y transports tls/ssl.
selectedStrict=socket con streams=true	Es correcto: strict excluye streams.

14. Verificación en 1.1.0

El checkout 1.1.0 tiene una superficie de verificación explícita y más pequeña que el árbol histórico de desarrollo 1.0. No presentes informes históricos de 10/14 gates como evidencia de 1.1.0.

Verificación del consumidor

Tras instalar, comprueba la plataforma real, inspecciona capacidades y ejecuta una petición HTTPS simple en el SAPI objetivo.

```
composer check-platform-reqs --no-dev
```

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

Verificación del checkout

Desde la raíz del repositorio, la guía actual documenta estos checks reproducibles:

```
composer validate --strict --no-check-lock
composer test
php tests/composer-consumer.php
find src tests -name '*.php' -print0 | xargs -0 -n1 php -l
php -l autoload.php
git diff --check
```

Qué prueban estos checks

- composer validate comprueba metadata y consistencia del paquete.
- composer test ejecuta el regression runner sin dependencias.
- tests/composer-consumer.php comprueba rechazo de PHP 8.5.11, instalación simulada 8.5.12, smoke con vendor/autoload.php y contenido real del archive Composer.
- Los checks de sintaxis cubren src/tests/autoload.php actuales.

Qué NO prueban

- No equivalen a la suite histórica completa de integración cURL/streams/TLS.
- No ejecutan la antigua fire matrix ni el panel QA de navegador; esas herramientas no están incluidas en el checkout actual.
- Por sí solos no prueban instalación remota desde GitHub o Packagist.

15. Distribución Composer y contenido del paquete

1.1.0 está estructurado como librería Composer normal. El requisito runtime del consumidor es solo PHP ^8.5.12; el tooling de contribución queda en require-dev.

- PSR-4: AbsolutOffice\XCurl\ -> src/.
- No hay version en composer.json raíz; las versiones estables proceden de tags VCS.
- Los archives Composer excluyen índices locales, caches, vendor, ficheros de entorno, PDFs y material de claves.
- LICENSE, COPYING, código runtime y tests de regresión incluidos permanecen distribuidos según los checks actuales.
- Los manuales PDF guardados en el repositorio quedan excluidos intencionadamente del paquete Composer por las reglas archive/export actuales.

```
"require": {
    "php": "^8.5.12"
},
"autoload": {
    "psr-4": {
        "AbsolutOffice\\XCurl\\": "src/"
    }
}
```

16. Mapa de API pública de la línea 1.x

La referencia API 1.0 sigue siendo el baseline histórico de compatibilidad para la línea 1.x. El código 1.1.0 conserva los tipos públicos centrales y añade información de propiedad mediante HttpResponse::targetCreated.

Tipo	Rol	Recordar
HttpTransportFactory	Factory/diagnóstico	create, createStrict, createNamed, diagnostics, selected*
HttpTransportInterface	Contrato transport	request(...) -> HttpResponse
CurlHttpTransport	Transport concreto	cURL
StreamHttpTransport	Transport concreto	Streams HTTPS de PHP
SocketHttpTransport	Transport concreto	Socket TLS directo
HttpRequest	Value object	method, url, headers, body
HttpResponse	Value object	statusCode, headers, body, bytes, targetCreated
HttpTransportException	Excepción transport	transient + TransportError
TransportError	Enum transport	12 categorías
Client\HttpClient	Orquestador de policy	send() preferido; request() wrapper
Client\RequestOptions	Opciones	timeoutSeconds, targetPath, deadlineSeconds
Client\RetryPolicy	Política retry	métodos, status, backoff, Retry-After
Client\RedirectPolicy	Política redirect	límite + strip de credenciales cross-origin
Client\HttpClientResult	Resultado	response, finalUrl, history, redirects, retries
Client\HttpClientAttempt	Intento	sequence, method, url, action, status/error/delay/nextUrl
Client\HttpClientAction	Enum historial	RETURNED, REDIRECTED, RETRIED, FAILED
Client\HttpClientException	Excepción cliente	ClientError + history
Client\ClientError	Enum cliente	5 categorías
ClockInterface	Contrato extensión/test	monotonicSeconds, unixTime
SleeperInterface	Contrato extensión/test	sleepMilliseconds

Superficie interna

Internal*, RequestValidator, SystemClock/SystemSleeper y helpers de capacidad son detalles de implementación/test; los consumidores normales no deben construir lógica de negocio alrededor de ellos.

17. Recetas copiables

Patrones pequeños para casos frecuentes.

A. GET simple

Usa HttpClient + HttpRequest + RequestOptions y revisa status/header/body.

```
require_once __DIR__ . '/vendor/autoload.php';

use AbsolutOffice\XCurl\HttpTransportFactory;

$transport = HttpTransportFactory::create();
$response = $transport->request(
    'GET',
    'https://example.com/',
    [],
    30,
);

echo $response->statusCode, PHP_EOL;
```

B. POST JSON sin retry automático

POST no es retryable por defecto; pasa body JSON raw y Content-Type.

```
$payload = json_encode($data, JSON_THROW_ON_ERROR);
$result = $client->send(
    new HttpRequest(
        'POST',
        'https://example.com/api/items',
        ['Content-Type: application/json'],
        $payload,
    ),
    new RequestOptions(timeoutSeconds: 30, deadlineSeconds: 60),
);
```

C. PUT binario con límites

Usa createStrict(maxResponseBytes: ..., maxRequestBytes: ...) si necesitas framing estricto.

```
$transport = HttpTransportFactory::createStrict(
    maxResponseBytes: 4 * 1024 * 1024,
    maxRequestBytes: 16 * 1024 * 1024,
);
```

D. Descarga grande a temporal

Usa targetPath único; valida y promueve el fichero tras éxito.

```
$tmp = sys_get_temp_dir() . '/download-' . bin2hex(random_bytes(8)) . '.tmp';

$result = $client->send(
    new HttpRequest('GET', 'https://example.com/file.dat'),
    new RequestOptions(
        timeoutSeconds: 30,
        targetPath: $tmp,
        deadlineSeconds: 120,
    ),
);

// Validate size/checksum/business rules, then promote the file.
```

E. Desactivar redirects

Construye RedirectPolicy(maxRedirects: 0).

```
$client = new HttpClient(
    HttpTransportFactory::create(),
    redirectPolicy: new RedirectPolicy(maxRedirects: 0),
);
```

F. Retry de POST solo si tu API es segura

Añade POST a methods únicamente con garantía real de idempotencia.

```
$retry = new RetryPolicy(maxRetries: 1, methods: ['GET', 'HEAD', 'POST']);
$client = new HttpClient(HttpTransportFactory::create(), $retry);
```

18. Anti-patrones

Evita estos errores de integración.

No hagas	Haz esto
“Si cURL falla, pruebo streams manualmente.”	Mantén el transport seleccionado; HttpClient reintenta solo si el fallo del mismo transport es transient y la policy lo permite.
Meter URLs/proveedores/credenciales/reglas de negocio en xCurl.	Mantenlos en la aplicación consumidora.
Activar follow redirects en el backend.	Deja redirects a HttpClient para política consistente.
Enviar Transfer-Encoding de request.	Envía body normal; xCurl usa Content-Length exacto.
Decidir lógica por texto de excepción.	Usa TransportError/ClientError, transient e history.
Usar createNamed() como selección normal.	Usa create()/createStrict(); createNamed() es para diagnóstico/test.
Reutilizar o sobrescribir target de descarga.	Usa temporal nuevo para cada operación.
Pensar que timeout=30 son exactamente 30.000 s en todos los backends.	Trátalo como safety timeout; usa deadline para presupuesto global.

19. Troubleshooting por síntomas

Empieza por el síntoma y diagnostics(); no cambies transports al azar.

Síntoma	Comprobar	Motivo
“No usable HTTPS transport”	HttpTransportFactory::diagnostics()	Revisa cURL, allow_url_fopen/wrapper https y OpenSSL tls/ssl.
Funciona CLI, falla web	Compara diagnostics() en ambos SAPI	Revisa php.ini y extensiones distintas.
Fallo TLS	TransportError::TLS + CA/hostname	Revisa SAN, cadena y trust bundle.
Target rechazado antes del request	¿La ruta ya existe o es symlink?	Usa temporal nuevo y único.
La limpieza del target detiene la operación	ClientError::TARGET_CLEANUP	Apareció un fichero no propio o no pudo eliminarse con seguridad.
No reintenta POST	RetryPolicy methods	Default solo GET/HEAD.
No sigue redirect	RedirectPolicy maxRedirects + Location	0 lo deshabilita; Location debe resolver a HTTPS.
Deadline antes de maxRetries	deadlineSeconds + delays	Correcto si no cabe otro intento completo.
Streams acepta algo que strict rechaza	create() vs createStrict()	Correcto: strict excluye streams.

20. Checklist de despliegue en un consumidor

El despliegue normal de 1.1.0 es Composer-first.

1. Requerir absolutoffice/xcurl:^1.1 en la aplicación consumidora.
2. Desplegar el árbol vendor generado por Composer.
3. Cargar vendor/autoload.php una sola vez desde el bootstrap.
4. Elegir create() o createStrict() según requisitos de framing.
5. Usar HttpClient::send() salvo que necesites deliberadamente una única llamada de transport.
6. Definir timeout y, para operaciones con varios intentos, deadline.
7. Definir límites de request/response cuando el caso lo necesite.
8. Mantener credenciales, endpoints, checksums, schemas y reglas de negocio fuera de xCurl.
9. Para descargas, usar temporal nuevo y validar antes de la promoción final.
10. Registrar enums de error e history, no depender del texto del mensaje.
11. Opcionalmente ejecutar composer check-platform-reqs --no-dev en el runtime real.

Nota de producción

El servidor de producción necesita el árbol vendor desplegado y su autoloader generado. El ejecutable composer puede quedarse en CI/build si las dependencias se instalan antes del despliegue.

21. Compatibilidad y mantenimiento

xCurl 1.1.0 requiere PHP ^8.5.12. La referencia 1.0 sigue siendo el baseline documentado para la línea 1.x, mientras 1.1.0 añade cambios compatibles de distribución y propiedad segura de descargas.

Cambio	Tratamiento
Bugfix interno sin ruptura pública	Puede entrar en patch/minor tras checks actuales y revisión de compatibilidad.
Adición pública backward-compatible	Una minor es apropiada cuando la API/comportamiento se amplía de forma compatible.
Cambio que rompe firmas/semántica pública existente	Requiere decisión explícita de versionado semántico.
Cambio de helper interno/test	No es contrato consumidor, pero deben pasar las regresiones.
Referencias históricas de QA	Conservar como archivo; no reutilizar como evidencia actual de 1.1.0.

22. Qué fuente manda cuando hay dudas

Este manual explica. El código y la metadata actual de release/paquete definen el comportamiento del checkout 1.1.0.

Fuente	Usa
src/	Implementación runtime exacta.
composer.json	Nombre, PHP, PSR-4, tooling dev y reglas de archive.
README.md	Resumen operativo compacto.
docs/INSTALLATION.md	Rutas de instalación y comandos de verificación actuales.
docs/releases/1.1.0.md	Cambios y límites de validación de la release.
tests/regression.php	Regresiones incluidas sin dependencias.
tests/composer-consumer.php	Checks de consumidor Composer y archive.
docs/reference/1.0.0/API-1.0.0.md	Referencia histórica de compatibilidad API 1.0 para la línea 1.x.

Cierre

Mantén separadas las capas: la aplicación decide negocio; HttpClient decide policy; HttpTransportInterface hace una petición; HttpTransportFactory selecciona el backend disponible.

Apéndice A. Chuleta de una página

Necesito...	Haz...
Instalar	composer require absolutoffice/xcurl:^1.1
Uso normal	new HttpClient(HttpTransportFactory::create()) + send(HttpRequest, RequestOptions)
Framing estricto	HttpTransportFactory::createStrict()
Una petición sin policy	$transport->request(...)
Descargar	RequestOptions(targetPath: $tmp), donde $tmp no existe
Body raw	HttpRequest(..., body: $payload); null != ""
Límites	create(maxResponseBytes: ..., maxRequestBytes: ...)
Retries	RetryPolicy; default GET/HEAD, maxRetries=2
Redirects	RedirectPolicy; default maxRedirects=5
Presupuesto total	RequestOptions(deadlineSeconds: ...)
Diagnosticar backend	HttpTransportFactory::diagnostics()
Fallo transport	HttpTransportException: error + transient
Fallo cliente	HttpClientException: error + history
Verificar runtime	composer check-platform-reqs --no-dev
Regresión checkout actual	composer test + php tests/composer-consumer.php

Orden de selección

Modo	1	2	3
create()	cURL	streams	socket
createStrict()	cURL	socket	-

Redirects

301	302	303	307	308
preserva	preserva	GET salvo HEAD; elimina body si cambia el método	preserva	preserva